Compiladores



Geração de código

Geração de código

- A geração de código depende não apenas das características da linguagem fonte, mas também de informações detalhadas:
  - Da arquitetura alvo.
  - Da estrutura do ambiente de execução.
  - Do sistema operacional da máquina alvo.
- Envolve também tentativas de **otimizar** a velocidade e/ou tamanho do código alvo.
- Em razão da complexidade, um compilador tipicamente quebra essa fase em vários passos, que usam diversas estruturas de dados intermediárias, e frequentemente requerem alguma forma de **código intermediário**.

Código intermediário e estruturas de dados para geração de código



Código intermediário e estruturas de dados para geração de código

- Uma estrutura de dados que represente o programa-fonte durante a tradução é denominada **representação intermediária**, ou **IR**.
- Apesar de a árvore sintática abstrata ser uma representação perfeitamente adequada para a geração de código, em nada se assemelha ao código alvo.
- O programador de um compilador deve:
  1. Gerar uma nova forma de representação intermediária, a partir da árvore sintática, que se assemelhe melhor ao código alvo.
  2. Gerar o código alvo a partir dessa nova representação.
- Essa representação intermediária semelhante ao código alvo é denominada **código intermediário**.
- O código intermediário pode também ser útil para facilitar o redirecionamento de um compilador para outros alvos.

Código de três endereços

Código de três endereços

- A instrução mais básica do código de três endereços tem a seguinte forma geral:

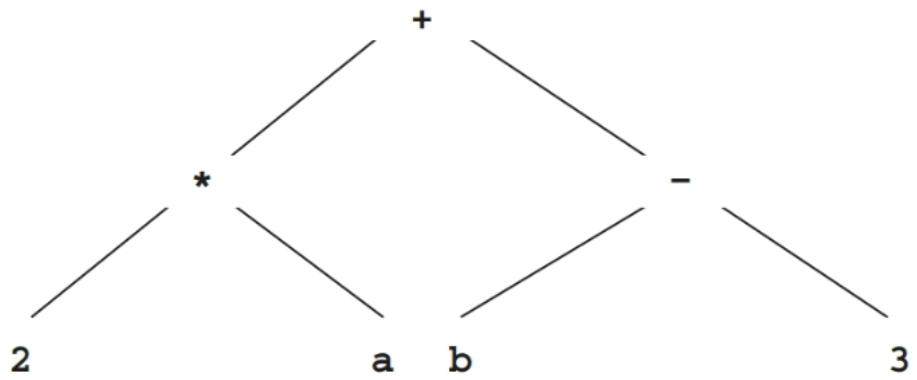
  x = y *op* **z**

- Expressa a aplicação do operador ***op*** aos valores de ***y*** e ***z*** e a atribuição desse valor como novo valor de ***x***.

- O nome "***código de três endereços***" advém dessa forma de instrução.
  - Em geral cada um dos nomes x, y e z representa um ***endereço*** na memória.
  - **y** e **z** podem representar endereços, constantes ou literais sem endereços na execução.

Código de três endereços

- Exemplo: considere a expressão aritmética `2*a+(b-3)`.
- com a árvore sintática:



- O código de três endereços correspondente é

```
t1 = 2 * a
t2 = b - 3
t3 = t1 + t2
```

Código de três endereços

- O código de três endereços requer que o compilador gere nomes para os temporários.
  - Esses temporários correspondem aos nós interiores da árvore sintática.
- A forma como esses temporários devem ser alocados na memória não é especificada por esse código.
  - Em geral são atribuídos a registradores.
  - Mas também podem ser armazenados na memória ou eliminados por otimizações.
- A forma única de código de três endereços vista não é suficiente para representar todas as características de uma linguagem.
  - Por exemplo, operadores unários como a negação requerem uma variação do código de três endereços que contenha apenas dois endereços.

```
t2 = -t1
```

Código de três endereços

- Exemplo de tradução de um programa em linguagem TINY.

```
read x;
if 0 < x then
  fact := 1;
  repeat
    fact := fact * x;
    x:=x-1
  until x = 0;
  write fact
end
```

```
read x
t1 = x > 0
if_false t1 goto L1
fact = 1
label L2
t2 = fact * x
fact = t2
t3 = x - 1
x = t3
t4 = x == 0
if_false t4 goto L2
write fact
label L1
halt
```

Estruturas de dados para a implementação do código de três endereços

Estruturas de dados para a implementação do código de três endereços

- O código de três endereços normalmente não é implementado em forma textual como no exemplo anterior.
- Cada instrução é implementada como uma estrutura de registros com diversos campos.
- A sequência de instruções é implementada como uma matriz ou lista ligada, que pode ser armazenada na memória ou escrita e lida a partir de arquivos temporários.

Estruturas de dados para a implementação do código de três endereços

- Para implementação do código de três endereços são necessários quatro campos, conhecidos como **quádrupla**:
  - Um para a operação.
  - Três para os endereços.
- Exemplo de quádruplas:

```
(rd,x,_,_)
(gt,x,0,t1)
(if_f,t1,L1,_)
(asn,1,fact,_)
(lab,L2,_,_)
(mul,fact,x,t2)
(asn,t2,fact,_)
(sub,x,1,t3)
(asn,t3,x,_)
(eq,x,0,t4)
(if_f,t4,L2,_)
(wri,fact,_,_)
(lab,L1,_,_)
(halt,_,_,_)
```

Estruturas de dados para a implementação do código de três endereços

- A quádruplas podem ser codificadas com as seguintes estruturas:

```
typedef enum {
  rd,gt,if_f,asn,lab,mul,sub,eq,wri,halt,. . .
} OpKind;
typedef enum {
  Empty,IntConst,String
} AddrKind;
typedef struct {
  AddrKind kind;
  union {
    int val;
    char *name;
  } contents;
} Address;
typedef struct {
  OpKind op;
  Address addr1, addr2, addr3;
} Quad;
```

Técnicas básicas para geração de código



Código intermediário ou código-alvo como atributo sintetizado

Código intermediário ou código-alvo como atributo sintetizado

- A geração de código intermediário pode ser vista como uma computação de atributos similar aos estudados.
- Exemplo: considere a gramática que representa um pequeno subconjunto de expressões e possui apenas as operações de soma (**+**) e atribuição (**=**):

```
exp → id = exp | aexp
aexp → aexp + fator | fator
fator → ( exp ) | num | id
```

Código intermediário ou código-alvo como atributo sintetizado

- A tabela expressa a gramática para a geração de código.
  - ***tacode*** → Atributo de código.
  - **++** → Concatenação de cadeias com inserção de linha.
  - **||** → Concatenação de cadeias com espaços.
  - ***name*** → Nome temporário para cálculo.
  - ***newtemp()*** → Cria nome temporário.

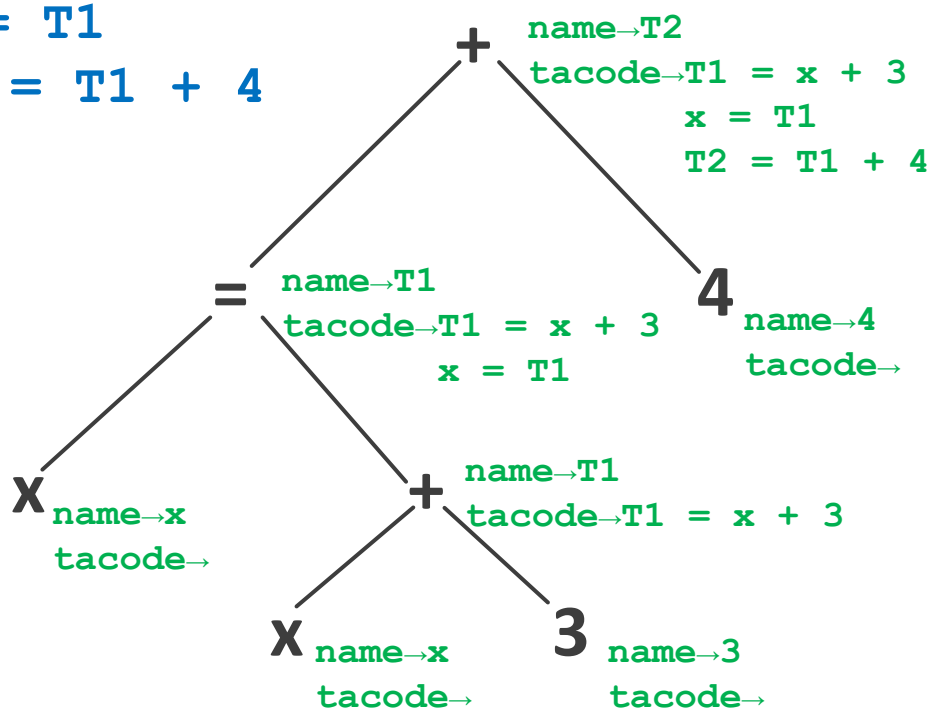
- Resolvendo a expressão **(x=x+3)+4**:

Regra Gramatical	Regras Semânticas
$exp_1 \rightarrow$ **id** = exp_2	exp_1 .name = exp_2 .name exp_1.tacode = exp_2 .tacode ++ **id**.strval || "=" || exp_2 .name
$exp \rightarrow aexp$	exp .name = aexp .name exp .tacode = aexp .tacode
$aexp_1 \rightarrow aexp_2$ + fator	$aexp_1$ .name = newtemp( ) $aexp_1$ .tacode = $aexp_2$ .tacode ++ fator .tacode ++ $aexp_1$ .name || "=" || $aexp_2$ .name || "+" || fator .name
$aexp \rightarrow fator$	aexp .name = fator .name aexp .tacode = fator .tacode
$fator \rightarrow$ (exp)	fator .name = exp .name fator .tacode = exp .tacode
$fator \rightarrow$ **num**	fator .name = **num**.strval fator .tacode = ""
$fator \rightarrow$ **id**	fator .name = **id**.strval fator .tacode = ""

Código intermediário ou código-alvo como atributo sintetizado

(x=x+3)+4

```
T1 = x + 3
x = T1
T2 = T1 + 4
```



Regra Gramatical	Regras Semânticas
$exp_1 \rightarrow$ **id** = exp_2	exp_1 .*name* = exp_2 .*name* exp_1.*tacode* = exp_2 .*tacode* ++ **id**.*strval* || "=" || exp_2 .*name*
$exp \rightarrow aexp$	*exp* .*name* = *aexp* .*name* *exp* .*tacode* = *aexp* .*tacode*
$aexp_1 \rightarrow aexp_2$ + *fator*	$aexp_1$.*name* = *newtemp*( ) $aexp_1$.*tacode* = $aexp_2$.*tacode* ++ *fator* .*tacode* ++ $aexp_1$.*name* || "=" || $aexp_2$.*name* || "+" || *fator* .*name*
$aexp \rightarrow fator$	*aexp* .*name* = *fator* .*name* *aexp* .*tacode* = *fator* .*tacode*
fator → (*exp*)	*fator* .*name* = *exp* .*name* *fator* .*tacode* = *exp* .*tacode*
fator → **num**	*fator* .*name* = **num**.*strval* *fator* .*tacode* = ""
fator → **id**	*fator* .*name* = **id**.*strval* *fator* .*tacode* = ""

Código intermediário ou código-alvo como atributo sintetizado

- A geração de código como computação de um atributo de cadeia sintetizado é útil, mas não é prática por diversas razões.
  1. Provoca uma grande quantidade de cópias de cadeias e desperdício de memória.
  2. Em geral é melhor gerar pequenos trechos de código à medida que ocorre a geração de código e gravar esses trechos em um arquivo ou inseri-los em uma estrutura de dados.
  3. A geração de código em geral depende grandemente dos atributos herdados, e isso complica muito as gramáticas de atributos.
- Consideraremos técnicas mais diretas de geração de código.

Geração de código-alvo a partir de código intermediário

Geração de código-alvo a partir de código intermediário

- Se um compilador gerar código intermediário diretamente durante a análise sintática ou a partir de uma árvore sintática, deve ocorrer uma passada adicional pelo código intermediário para gerar o código alvo final.
- A passada final da geração de código deve fornecer:
  - Localizações das variáveis e dos temporários.
  - O código necessário para manter o ambiente de execução.
- Uma questão importante é a alocação dos registradores e a manutenção das informações sobre o uso dos registradores.

Geração de código-alvo a partir de código intermediário

- Normalmente a geração de código a partir de código intermediário requer uma das ou as duas seguintes técnicas padrão:
  - **Expansão de macros** → Requer a substituição de cada tipo de instrução de código intermediário por uma sequência equivalente de instruções de código alvo.
  - **Simulação estática** → Requer uma simulação direta dos efeitos do código intermediário e a geração de código alvo correspondente a esses efeitos.

Geração de código para referências e estruturas de dados



Cálculos de endereços

Cálculos de endereços

- Vimos como o código intermediário pode ser gerado a partir de expressões aritméticas simples e atribuições.
- Todos os valores básicos eram constantes ou variáveis simples.
  - As variáveis simples foram identificadas apenas por nome.
- A tradução para o código alvo exige que esses nomes sejam substituídos pelos endereços efetivos que poderiam ser:
  - Registradores.
  - Endereços absolutos de memória (para globais).
  - Deslocamentos de registros de ativação (para locais).

Cálculos de endereços

- Há situações que exigem cálculos para localizar o endereço.
  - Esses cálculos devem ser expressos diretamente.
  - Ocorrem na indexação de matrizes, nos campos de registros e nas referências de ponteiros.
- No código de três endereços não existe tanta necessidade de novas operações.
  - As operações aritméticas usuais podem ser usadas para computar endereços.
- Mas novas operações podem ser úteis mas de indicar os modos de endereçamento "*endereço de*" e "*indireto*".
  - Usaremos a notação equivalente da linguagem C "**&**" e "*****" para indicar esses modos de endereçamento.

Cálculos de endereços

- Exemplo: armazenar o valor constante 2 no endereço da variável x mais 10 bytes.

```
t1 = &x + 10
*t1 = 2
```

- A implementação desses novos modos de endereçamento exige que a estrutura de dados para o código de três endereços contenha novos campos.
- A estrutura de dados de quádrupla pode ser estendida para um campo ***AddrMode*** enumerado com valores possíveis ***None***, ***Address*** e ***Indirect***.

Referências de matrizes

Referências de matrizes

- Uma referência de matriz requer a indexação de uma variável de matriz por uma expressão para obter uma referência ou valor de um elemento único.
- Exemplo:

```
int a[SIZE], i, j;
...
a[i+1] = a[j*2] + 3;
```

- A indexação de **a** pela expressão **i+1** produz um endereço (alvo da atribuição), e a indexação de **a** pela expressão **j*2** produz o valor no endereço computado pelo tipo de elemento de **a** (no caso, int).
- Como as matrizes são armazenadas sequencialmente na memória, cada endereço deve ser computado a partir do **endereço de base** de **a** e de um deslocamento que dependa linearmente do valor do índice.

Referências de matrizes

- Para computar o deslocamento:
  1. Deve ser efetuado um ajuste no valor do índice se o intervalo de índices não começar por zero.
  2. O valor de índice ajustado deve ser multiplicado por um fator de escala, que é igual ao tamanho de cada elemento da matriz na memória.
  3. O índice ajustado resultante é somado ao endereço de base para obter o endereço final do elemento da matriz.
- O endereço da referência da matriz de inteiros em C `a[i+1]` é:

```
&a + (i+1) * sizeof(int)
```

- O endereço de um elemento de matriz `a[t]` em qualquer linguagem é:

```
endereço_base(a) + (t - limite_inferior(a)) * tamanho_elemento(a)
```

Referências de matrizes

CÓDIGO DE TRÊS ENDEREÇOS PARA REFERÊNCIAS A MATRIZES

- Uma forma possível de expressar referências a matrizes no código de três endereços é introduzir duas novas operações:
  - Uma que captura o valor de um elemento de matriz.
    - t2 = a[t1]
  - Uma que atribui um valor ao endereço de um elemento de matriz.
    - a[t2] = t1
- Exemplo: a declaração em código-fonte:

```
a[i+1] = a[j*2] + 3;
```

- seria traduzida nas instruções de três endereços:

```
t1 = j * 2
t2 = a[t1]
t3 = t2 + 3
t4 = i+1
a[t4] = t3
```

Referências de matrizes

- Ainda é preciso introduzir modos de endereçamento que suportem estas operações.
- Também é possível escrever diretamente as computações de endereços de um elemento de matriz em código de três endereços.
- O código de três endereços para a atribuição **t2 = a[t1]** é:

```
t3 = t1 * elem_size(a)
t4 = &a + t3
t2 = *t4
```

- O código de três endereços para a atribuição **a[t2] = t1** é:

```
t3 = t2 * elem_size(a)
t4 = &a + t3
*t4 = t1
```

Referências de matrizes

- A declaração em código-fonte `a[i+1] = a[j*2] + 3` é traduzida para as instruções de três endereços:

```
t1 = j * 2
t2 = t1 * elem_size(a)
t3 = &a + t2
t4 = *t3
t5 = t4 + 3
t6 = i + 1
t7 = t6 * elem_size(a)
t8 = &a + t7
*t8 = t5
```

Referências de matrizes

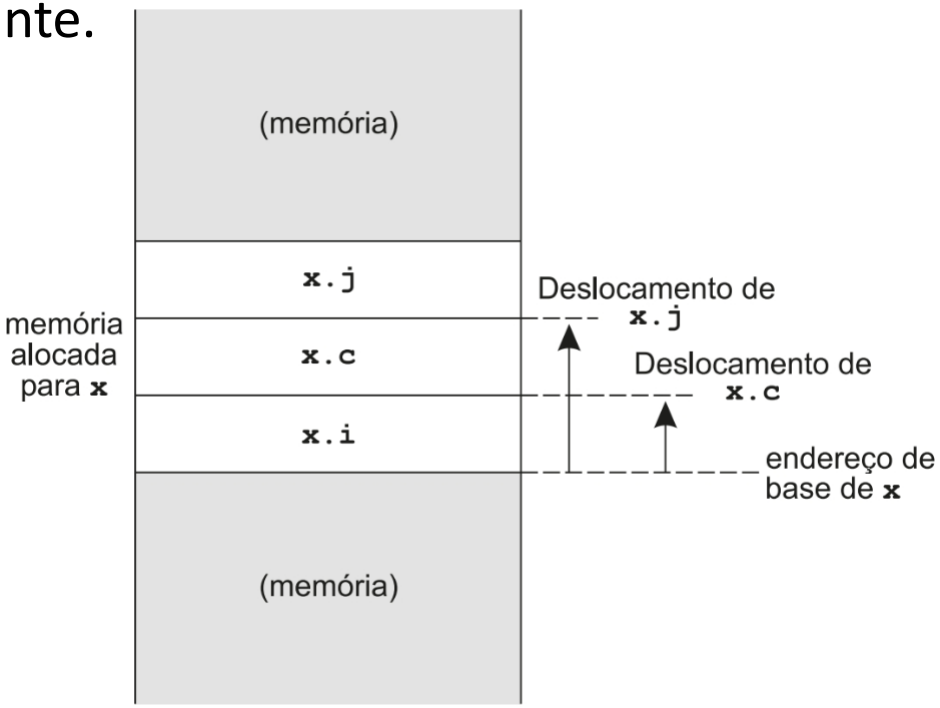
- **MATRIZES MULTIDIMENSIONAIS**
- Um complicador para a computação de endereços de matrizes é a existência de matrizes com múltiplas dimensões.
- Essas matrizes podem ser indexadas parcialmente, levando a matrizes de dimensões menores.
- Exemplo:
  - Na matriz `int a[15][10]` a expressão `a[i]` indexa parcialmente `a`, levando a uma matriz unidimensional de inteiros.
- O endereço de uma variável de tipo matriz pode ser computado com a aplicação recursiva das técnicas descritas.

Estrutura de registros e referências de ponteiros

Estrutura de registros e referências de ponteiros

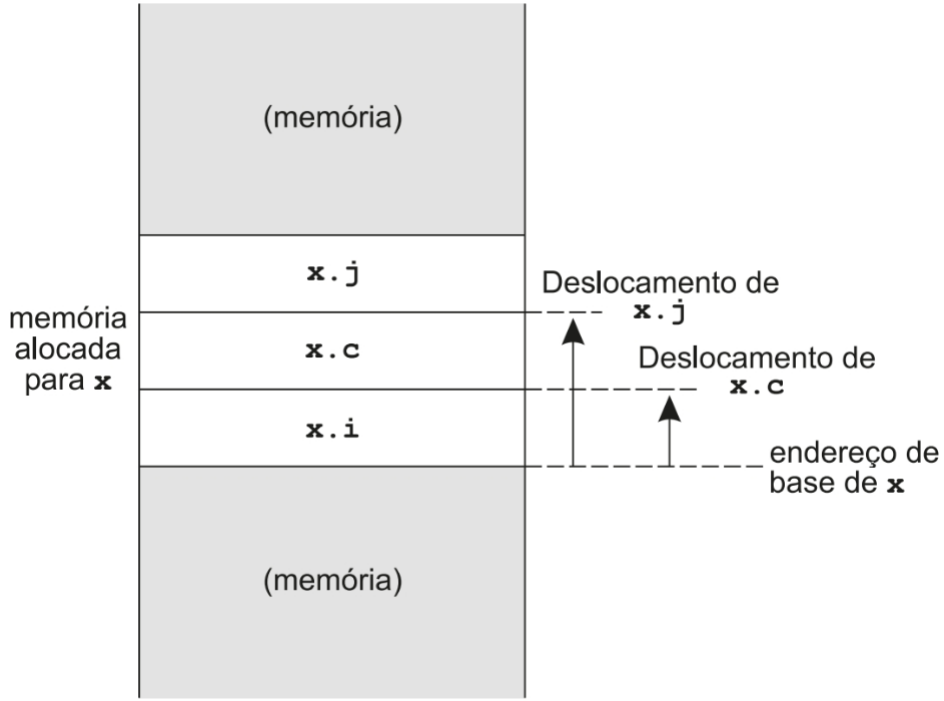
- A computação do endereço de um campo de registro ou estrutura é semelhante ao da computação de um endereço de matriz indexada.
  1. O endereço de base da variável de estrutura é computado.
  2. O deslocamento (em geral fixo) do campo com nome é encontrado.
  3. Os dois são adicionados para obter o endereço resultante.
- Exemplo: as declarações em C na listagem abaixo normalmente são colocadas em memória como:

```
typedef struct rec {
   int i;
   char c;
   int j;
} Rec;
...
Rec x;
```



Estrutura de registros e referências de ponteiros

- Observe que:
  1. Os campos são alocados linearmente.
  2. O deslocamento de cada campo é uma constante.
  3. O primeiro campo (x.i) tem deslocamento zero.
  4. Os deslocamentos dos campos dependem dos tamanhos dos diferentes tipos de dados na máquina alvo.
- Precisamos introduzir uma nova função que retorne o deslocamento de um campo, dada uma variável de estrutura e um nome de campo.
- Essa função recebe o nome de ***field_offset***, e é escrita com dois parâmetros:
  - O nome da variável.
  - O nome do campo.
- ***field_offset(x,j)*** retorna o deslocamento de **x.j**.
- Essa função pode ser fornecida pela tabela de símbolos.



Estrutura de registros e referências de ponteiros

- Estruturas de registros são utilizadas em conjunto com ponteiros e memória dinâmica, para implementar estruturas de dados dinâmicas como listas e árvores.

- Um ponteiro simplesmente estabelece um nível adicional de referência indireta.

Estrutura de registros e referências de ponteiros

- Uma atribuição de campo como a declaração em C

```
x.j = x.i;
```

- pode ser traduzida no código de três endereços

```
t1 = &x + field_offset(x,j)
t2 = &x + field_offset(x,i)
*t1 = *t2
```

Estrutura de registros e referências de ponteiros

- Suponha que x seja declarada como um ponteiro para um inteiro pela declaração em C

```
int *x;
```

- Suponha que i seja uma variável inteira normal. Então, a atribuição em C

```
*x = i;
```

- pode ser traduzida na instrução de três endereços

```
*x = i
```

- e a atribuição

```
i = *x;
```

- na instrução de três endereços

```
i = *x
```

Estrutura de registros e referências de ponteiros

- Exemplo: considere uma estrutura de dados de árvore e uma declaração de variável em C:

```
typedef struct treeNode {
  int val;
  struct treeNode *lchild, *rchild;
} TreeNode;
...
TreeNode *p;
```

- Considere agora duas atribuições típicas

```
p->lchild = p;
p = p->rchild;
```

- Essas declarações são traduzidas no código de três endereços

```
t1 = p + field_offset(*p, lchild)
*t1 = p
t2 = p + field_offset(*p, rchild)
p = *t2
```

Geração de código para declarações de controle e expressões lógicas



Geração de código para declarações de controle e expressões lógicas

- A geração de código intermediário para declarações de controle requer a geração de **rótulos**.
  - Indicação de endereços no código alvo para onde ocorrem os saltos.

Geração de código para declarações if e while

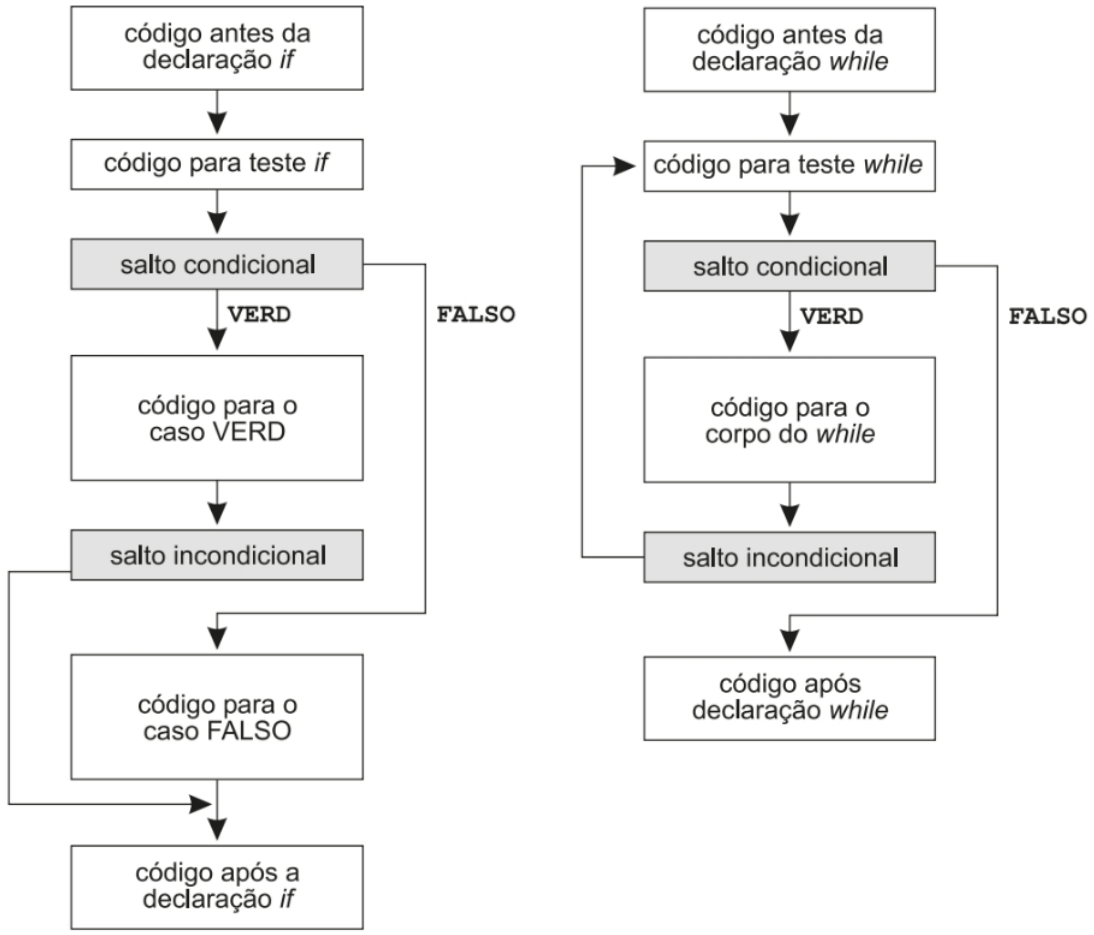
Geração de código para declarações if e while

- Considere as declarações ***if*** e ***while*** na forma:

```
if-decl → if ( exp ) decl | if ( exp ) decl
else decl
while-decl → while ( exp ) decl
```

- Compiladores geram código para as declarações ***if*** e ***while*** em uma forma padrão que permite o uso eficiente de um subconjunto dos saltos possíveis considerando uma arquitetura alvo.

- As possíveis organizações para cada uma dessas declarações são:



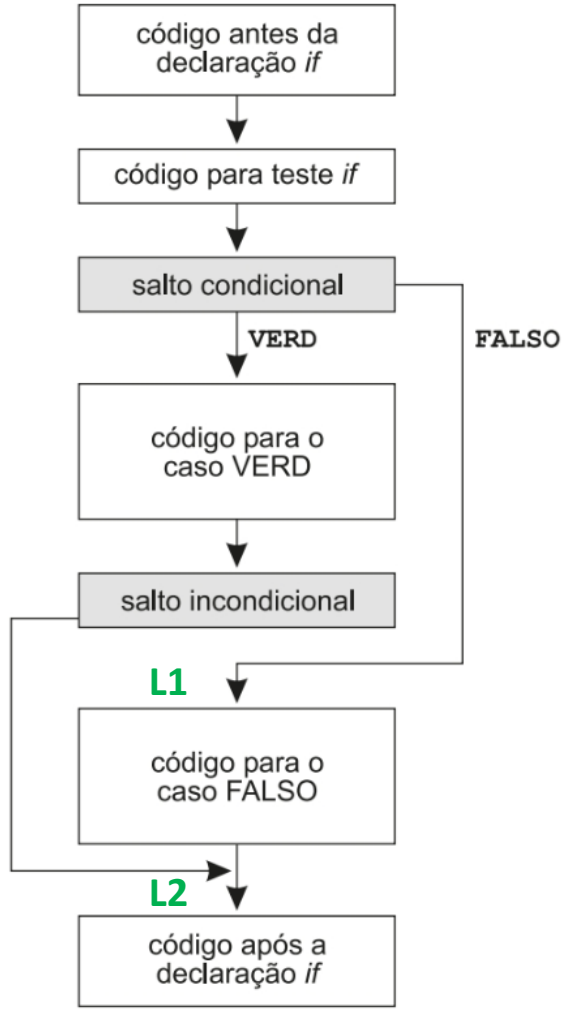
Geração de código para declarações if e while

- Assumimos que o gerador de código gera uma sequência de rótulos com nomes como L1, L2 e assim por diante.
- Para a declaração

```
if ( E ) S1 else S2
```

- o seguinte padrão de código é gerado:

```
<código para avaliar E para t1>
if_false t1 goto L1
<código para S1>
goto L2
label L1
<código para S2>
label L2
```



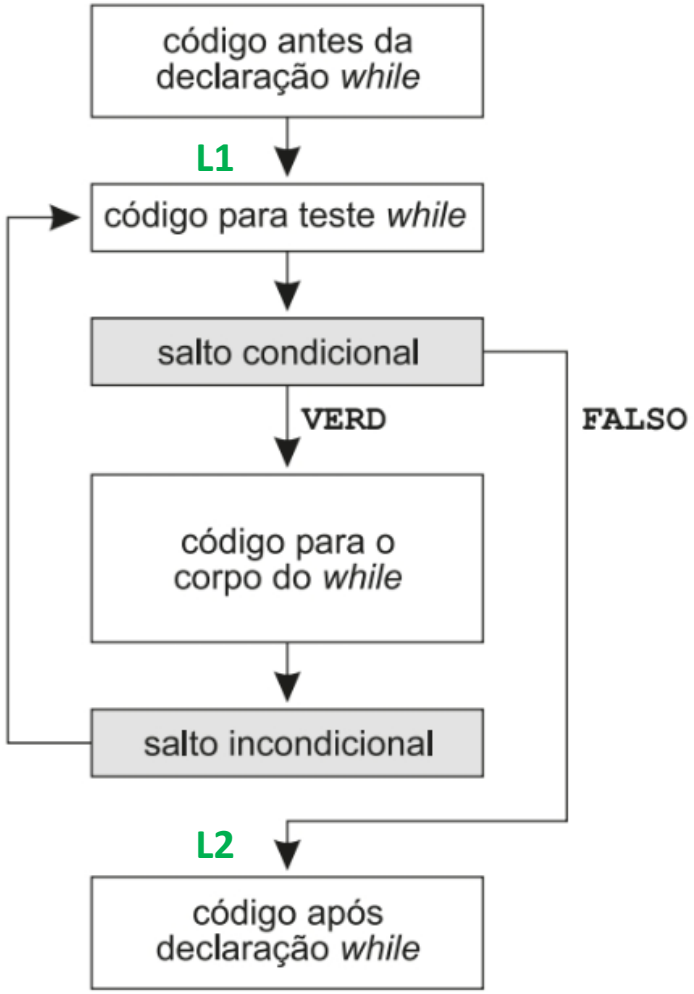
Geração de código para declarações if e while

- Uma declaração ***while*** da forma

```
while ( E ) S
```

- produziria o padrão de código:

```
label L1
<código para avaliar E para t1>
if_false t1 goto L2
<código para S>
goto L1
label L2
```



Geração de rótulos e ajuste retroativo

Geração de rótulos e ajuste retroativo

- Em alguns casos, os saltos para um rótulo podem ser gerados antes da definição do próprio rótulo.

- Um método padrão que gere esses saltos para adiante está em deixar um espaço em branco no código em que o salto deve ocorrer ou gerar uma instrução provisória de salto para uma localização fictícia.
  - Quando o salto é determinado a localização é usada para ajustar retroativamente o código que ficou faltando.
    - Técnica conhecida como ***backpatching*** (**ajuste retroativo**).

Geração de código de expressões lógicas

Geração de código de expressões lógicas

- Se o código intermediário tiver um tipo de dados booleano e operações lógicas como **and** e **or**, o valor de uma expressão booleana pode ser computado em código intermediário exatamente da mesma forma que uma expressão aritmética.
- Mesmo nesse caso a tradução em código alvo geralmente requer que valores booleanos sejam representados aritmeticamente.
  - A maioria das arquiteturas não apresenta um tipo de dados booleano interno.
  - A forma padrão é representar o valor booleano **falso como zero** e o **verdadeiro como 1**.
  - Os **operadores de bits** padrão **and** e **or** podem ser usados para computar os valores de uma expressão booleana na maioria das arquiteturas.

Geração de código de expressões lógicas

- Um uso adicional dos saltos é requerido se as operações lógicas forem um **curto-circuito**, como ocorre em C.
  - Em uma operação lógica com curto-circuito o segundo operando só é avaliado quando seu valor pode ser necessário para determinar o resultado da expressão
- Por exemplo, se **a** for uma expressão booleana computada como falsa, a expressão booleana **a and b** pode ser determinada de imediato como falsa sem avaliar **b**.
- Da mesma forma se **a** for verdadeira, então **a or b** pode ser determinada como verdadeira sem avaliar **b**.

Geração de código de expressões lógicas

- É comum escrever em C:

```
if ((p!=NULL) && (p->val==0)) ...
```

- onde a avaliação de p->val, quando p for nulo, teria comportamento indefinido.
- Operadores booleanos de curto-circuito são similares a declarações ***if***, exceto que retornam valores e frequentemente são definidos com base em expressões ***if*** como:

```
a and b ≡ if a then b else false
```

- e

```
a or b ≡ if a then true else b
```

- Para gerar código para curto-circuito precisamos usar saltos da mesma forma que no código para as declarações ***if***.

Geração de código para chamadas de procedimentos e funções



Geração de código para chamadas de procedimentos e funções

- A complexidade da descrição do código intermediário e do código alvo para esse mecanismo é aumentada em decorrência de diferentes máquinas alvo usarem mecanismos consideravelmente distintos para as ativações.
- Além disso as ativações dependerem da organização do ambiente de execução.
- É difícil obter uma representação de código intermediário que seja suficientemente geral para ser usada em qualquer arquitetura alvo ou ambiente de execução.

Código intermediário para procedimentos e funções

Código intermediário para procedimentos e funções

- Existem dois mecanismos que precisam ser descritos:
  - A **definição** de uma função/procedimento (**declaração**).
  - A **ativação** da função/procedimento.
- Uma **definição cria um nome de função**, parâmetros e código.
  - A função não é executada naquele ponto.
- Uma **ativação**:
  1. **Cria os valores** para os parâmetros (os **argumentos** da ativação).
  2. **Efetua um salto** para o código da função, que é executada e retorna um valor.

Código intermediário para procedimentos e funções

- O código intermediário para uma definição precisa conter:
  - Uma instrução que marque o início, ou **ponto de entrada**, do código para a função.
  - Uma instrução que marque o final, ou **ponto de retorno**, da função.
- Esquematicamente:

```
Instrução de entrada
<código para o corpo da função>
Instrução de retorno
```

- Uma ativação de função precisa:
  - De uma instrução para indicar o início da computação dos argumentos (em preparação para a ativação).
  - De uma instrução de ativação que indique o ponto em que os argumentos foram construídos e para onde pode ser efetuado o salto para o código da função:
- Esquematicamente:

```
Instrução de argumento de início da computação
<código para computar os argumentos>
Instrução de ativação
```

Código intermediário para procedimentos e funções

- Em código de três endereços, a instrução de entrada precisa dar um nome para o ponto de entrada do procedimento.
  - Similar à instrução **label**.
  - É uma instrução de um endereço denominada **entry**.

- A instrução de retorno é denominada **return**.
  - É uma instrução de um endereço.
  - Precisa dar o nome do valor de retorno, se houver algum.

Código intermediário para procedimentos e funções

- Exemplo: a definição da função em C

```
int f(int x, int y) {
  return x+y+1;
}
```

- é traduzida no código de três endereços:

```
entry f   // Define o ponto de entrada da função
t1 = x + y
t2 = t1 + 1
return t2   // Transfere t2 para o mecanismo de retorno
```

Código intermediário para procedimentos e funções

- No caso de uma ativação são necessárias três instruções distintas de três endereços:
  - Uma para indicar o início da computação de argumentos (**begin_args**).
  - Uma instrução usada repetidamente para especificar os nomes dos valores de argumentos (**arg**).
  - A instrução de ativação (**call**).

Código intermediário para procedimentos e funções

- Exemplo: a definição da função em C

```
int f(int x, int y) {
  return x+y+1;
}
```

- com ativação

```
f(2+3,4)
```

- é traduzida para o código de três endereços

```
begin_args
t1 = 2 + 3
arg t1
arg 4
call f
```

TM: uma máquina-alvo simples



TM: uma máquina-alvo simples

- Veremos um gerador de código para a linguagem TINY.
- Geraremos o código diretamente para uma máquina simples que pode ser simulada com facilidade.
- Essa máquina é denominada TM (TINY Machine - Máquina TINY).

Arquitetura básica da máquina TINY

Arquitetura básica da máquina TINY

- TM é composta por:
  - Uma memória de instruções de apenas leitura.
  - Uma memória de dados.
  - Um conjunto de oito registradores de uso geral.
    - Todos eles usam endereços inteiros não negativos começando pelo zero.

- O **registrador 7** é o **contador de programa (*program conter*)** e é o único registrador especial.

Arquitetura básica da máquina TINY

- Definições da TM:

```
#define IADDR_SIZE ... /* tamanho da memória de instruções
*/
#define DADDR_SIZE ... /* tamanho da memória de dados */
#define NO_REGS 8 /* quantidade de registradores */
#define PC_REG 7
Instruction iMem[IADDR_SIZE];
int dMem[DADDR_SIZE];
int reg[NO_REGS];
```

Arquitetura básica da máquina TINY

- TM efetua um ciclo convencional de captura-execução:

```
do
   /* captura */
   currentInstruction = iMem[reg[PC_REG]++];
   /*executa instrução corrente*/
   ...
while (!(halt||error)) ;
```

Arquitetura básica da máquina TINY

- Ao iniciar todos os registradores e a memória de dados são ajustados para zero.
- Em seguida carrega o valor do endereço legal mais alto em dMem[0].
  - Para indicar o início da pilha, que é alocada no endereço mais alto de memória.
- A TM passa a executar as instruções, começando por iMem[0].
- A máquina para quando é executada uma instrução HALT.

Arquitetura básica da máquina TINY

- Existem dois formatos básicos de instrução:
  - Instruções de somente-registrador (RO).
  - Instruções de registrador-memória (RM).
- Uma **instrução RO** tem o formato

```
opcode r,s,t
```

- onde os operandos **r**, **s** e **t** são registradores legais.
- São instruções de três endereços.
  - E todos os três endereços devem ser registradores.
- Todas as **instruções aritméticas** são limitadas a esse formato, assim como as **instruções de entrada/saída primitivas**.

Arquitetura básica da máquina TINY

Instruções RO

Formato	***opcode r,s,t***
Opcode	Efeito
HALT	interrompe a execução (ignora operandos)
IN	**reg[*r*]** ← valor inteiro lido da entrada padrão (***s*** e ***t*** são ignorados)
OUT	**reg[*r*]** → saída padrão (***s*** e ***t*** são ignorados)
ADD	**reg[*r*] = reg[*s*] + reg[*t*]**
SUB	**reg[*r*] = reg[*s*] - reg[*t*]**
MUL	**reg[*r*] = reg[*s*] * reg[*t*]**
DIV	**reg[*r*] = reg[*s*] / reg[*t*]** (pode gerar **ZERO_DIV**)

Arquitetura básica da máquina TINY

- Uma instrução RM tem o formato

```
opcode r,d(s)
```

- Onde **r** e **s** devem ser registradores legais e **d** deve ser um inteiro positivo ou negativo que represente um **deslocamento**.
- Essa instrução tem dois endereços:
  - O primeiro endereço é sempre um registrador.
  - O segundo é um endereço de memória é dado por `a = d + reg[s]`.

Arquitetura básica da máquina TINY

Instruções RM

Formato *opcode* `r,d,(s)`

(`a = d + reg[s]`; qualquer referência a `dMem[a]` gera `DMEM_ERR` se `a` < 0 ou se `a` ≥ `DADDR_SIZE`)

Opcode	Efeito
`LD`	`reg[r] = dMem[a]` (carrega `r` com valor de memória em `a`)
`LDA`	`reg[r] = a` (carrega e endereça `a` diretamente em `r`)
`LDC`	`reg[r] = d` (carrega constante `d` diretamente em `r`–`s`, que é ignorado)
`ST`	`dMem[a] = reg[r]` (armazena valor em `r` na posição de memória `a`)
`JLT`	`if (reg[r]<0) reg[PC_REG] = a` (salta para a instrução `a` se `r` for negativo, e de maneira similar para o que segue abaixo)
`JLE`	`if (reg[r]<=0) reg[PC_REG] = a`
`JGE`	`if (reg[r]>=0) reg[PC_REG] = a`
`JGT`	`if (reg[r]>0) reg[PC_REG] = a`
`JEQ`	`if (reg[r]==0) reg[PC_REG] = a`
`JNE`	`if (reg[r]!=0) reg[PC_REG] = a`

Arquitetura básica da máquina TINY

- Não há uma instrução de salto incondicional, mas ela também pode ser implementada por

```
LD 7, 0(1)
```

- que salta para a instrução cujo endereço é armazenado na memória, no ponto indicado pelo registrador 1.
- Instruções de salto condicional podem ser relativas à posição corrente no programa colocando ***PC*** como segundo registrador. Por exemplo:

```
JEQ 0,4(7)
```

- faz a TM saltar cinco instruções para a frente no código se o registrador zero for zero.

Simulador TM

Simulador TM

- O simulador TM aceita os arquivos de texto contendo as instruções TM com as seguintes convenções:
  1. Uma linha totalmente em branco é ignorada.
  2. Uma linha que começa com um asterisco é considerada um comentário e é ignorada.
  3. Qualquer outra linha precisa conter uma localização de instrução inteira seguida de uma vírgula e de uma instrução legal.
  4. Qualquer texto seguindo a instrução é considerado um comentário e é ignorado.

Simulador TM

- Exemplo de programa TM:

```
* Este programa recebe como entrada um inteiro, computa
* o seu fatorial se ele for positivo
* e imprime o resultado

0:      IN   0,0,0      r0 = read
1:      JLE  0,6(7)     if 0 < r0 then
2:      LDC  1,1,0        r1 = 1
3:      LDC  2,1,0        r2 = 1
                        * repete
4:      MUL  1,1,0         r1 = r1 * r0
5:      SUB  0,0,2         r0 = r0 - r2
6:      JNE  0,-3(7)      until r0 == 0
7:      OUT  1,0,0        write r1
8:      HALT 0,0,0      halt
* final do programa
```

Simulador TM

- Não há necessidade de as localizações aparecerem em ordem ascendente.
- Cada linha de entrada é efetivamente uma diretiva "*armazene essa instrução nessa localização*".
  - Essa propriedade do simulador TM facilita o ajuste retroativo dos saltos na ausência de rótulos simbólicos.
- O comando **g** indica "começar" (do inglês *go*),.
  - O programa começa a execução com o conteúdo corrente do PC.
- A lista completa de comandos do simulador pode ser obtida com o comando **h**. (*help*).

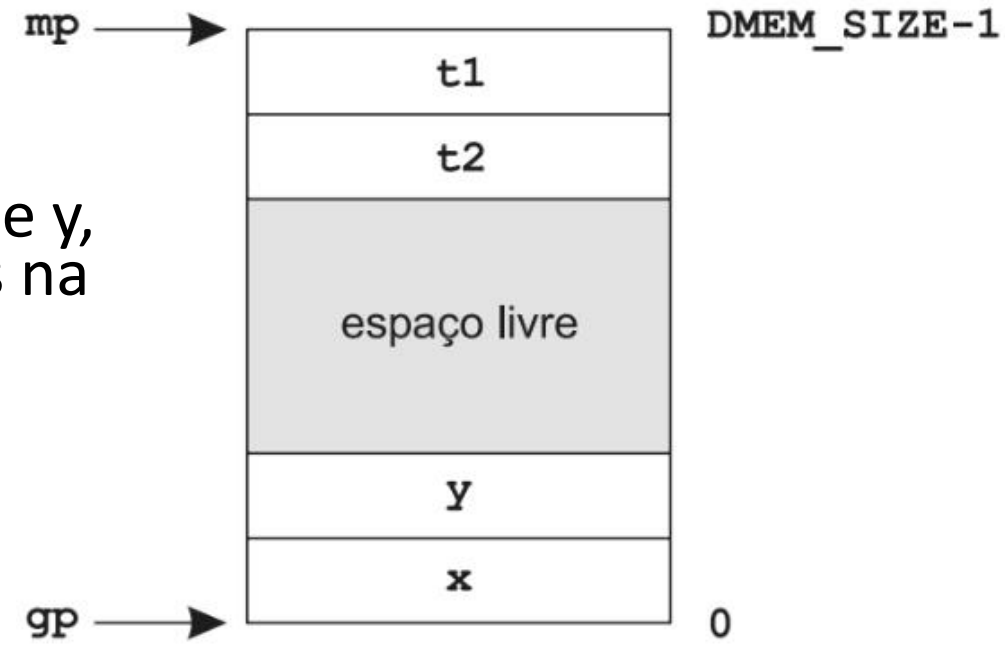
Gerador de código para a linguagem TINY



Interface TM do gerador de código TINY

Interface TM do gerador de código TINY

- O arquivo "**code.h**" possui definições para a máquina TINY.
  - Define **pc** como o **registrador 7**.
  - Define **mp** (ponteiro de memória) como o **registrador 6**.
    - Usado para acessar a memória de temporários (como uma pilha).
    - Localização mais alta da memória.
  - Define **gp** (ponteiro global) como o **registrador 5**.
    - Usado para acesso a variáveis em memória.
- Exemplo: se um programa tiver duas variáveis x e y, e houver dois valores temporários armazenados na memória, dMem ficará como na imagem.
  - Endereço de t1 é 0(mp).
  - Endereço de t2 é -1(mp).
  - Endereço de x é 0(gp).
  - Endereço de y é 1(gp).



Interface TM do gerador de código TINY

- Os outros dois registradores que serão usados pelo gerador de código são os registradores zero e 1, denominados **acumuladores**, e que recebem os nomes **ac** e **ac1**.
  - Os resultados das computações serão sempre colocados em ac.
- Os registradores 2, 3 e 4 não recebem nomes e nunca serão usados.
- Existem 7 funções de emissão de código.
- **`emitComment(...)`**
  - Imprime uma linha de comentario no arquivo de código.
- **`emitRO(...)`**
  `emitRM(...)`
  - Funções padrão de emissão de código, usadas para as classes de instruções RO e RM.

Interface TM do gerador de código TINY

- `emitSkip(...)`
  - Usada para pular diversas localizações que serão ajustadas retroativamente posteriormente.
- `emitBackup(...)`
  - Usada para ajustar a localização da instrução corrente em uma localização anterior para ajuste retroativo.
- `emitRestore(...)`
  - Usada para retornar a localização da instrução corrente para o valor anterior à ativação de emitBackup.
- `emitRM_Abs(...)`
  - Gerar o código em um salto de ajuste retroativo ou qualquer salto para uma localização de código retornada por uma ativação de emitSkip.
  - Transforma um endereço de código absoluto em endereço relativo a PC.

Interface TM do gerador de código TINY


code.c


code.h

Gerador de código TINY

Gerador de código TINY

- O gerador de código TINY é implementado no arquivo “**cgen.c**”.
- A função de entrada é

```
codeGen(TreeNode *syntaxTree, char *codefile);
```

- Durante sua execução ela:
  1. Gera alguns comentários e instruções (**prólogo padrão**) .
     - Comentário inicial com o nome do arquivo.
     - Ajusta o ambiente no início da execução.
  2. Ativa a função `cGen()` .
     - Recebe a árvore sintática.
     - Gera o código efetivo do programa.
  3. Emite uma instrução **HALT** para o final do programa.

Gerador de código TINY

- `void cGen(TreeNode *tree);`
- Função responsável pelo percurso da árvore sintática TINY, que possui formato:

```
typedef enum {StmtK,ExpK} NodeKind;
typedef enum {IfK,RepeatK,AssignK,ReadK,WriteK} StmtKind;
typedef enum {OpK,ConstK,IdK} ExpKind;

#define MAXCHILDREN 3
typedef struct treeNode {
   struct treeNode * child[MAXCHILDREN];
   struct treeNode * sibling;
   int lineno;
   NodeKind nodekind;
   union { StmtKind stmt; ExpKind exp;} kind;
   union { TokenType op;
           int val;
           char * name; } attr;
   ExpType type; /* para checagem de tipo das espressoes */
} TreeNode;
```

Gerador de código TINY

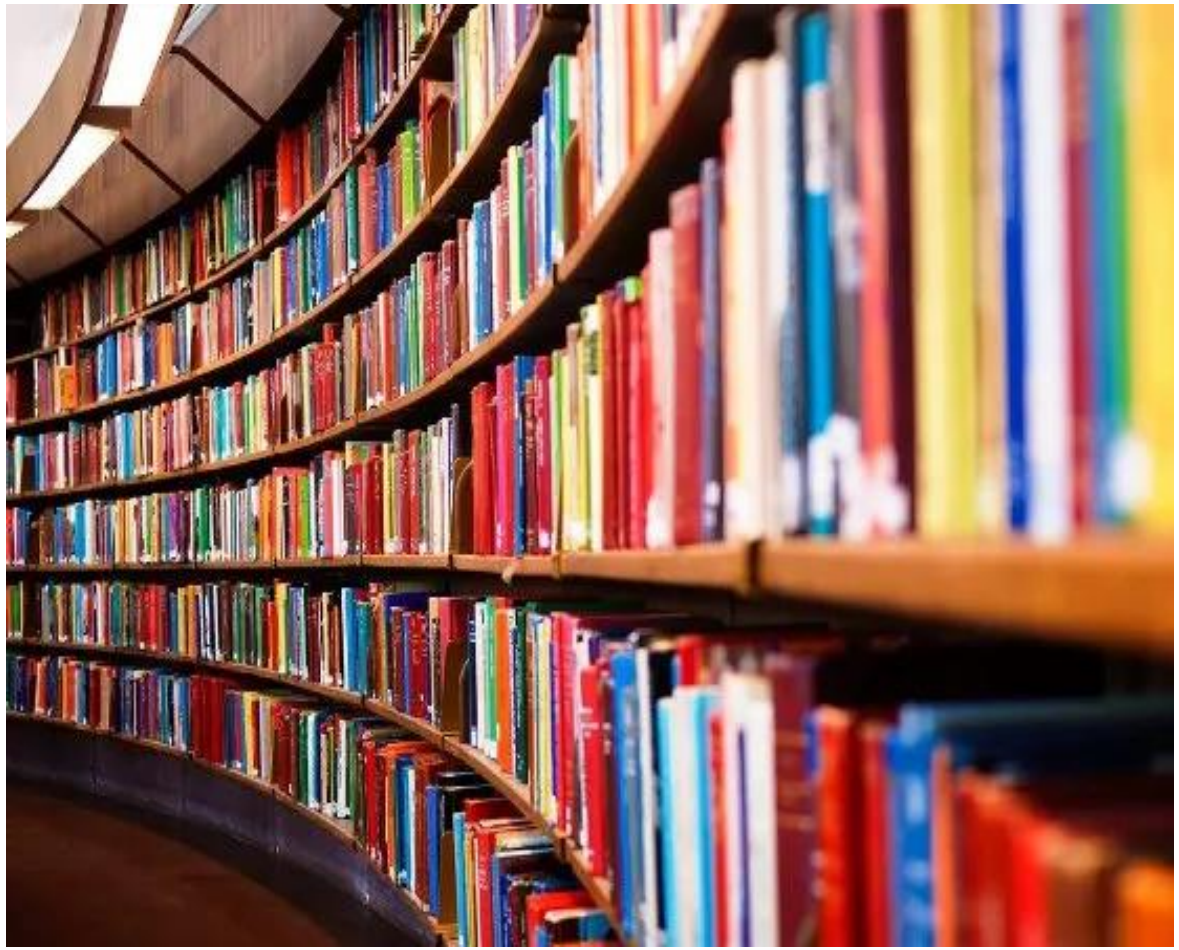
- Há dois tipos de nós de árvore:
  - **Nó de declaração** → Representa um dos cinco tipos diferentes de declaração TINY (***if***, ***repeat***, ***assign***, ***read*** ou ***write***).
  - **Nó de expressão** → Representa um dos três tipos de expressão (**identificador**, **constante inteira** ou **operador**).
- A função `cGen()` apenas testa se um nó é um nó de declaração ou de expressão, ativando a função apropriada.
  - `genStmt()` ou `genExp()`.
- Em seguida ativa a si mesma recursivamente para os irmãos.

Gerador de código TINY


cgen.c


cgen.h

Compiladores



Bibliografia

Bibliografia

- AHO, A. V., LAM, M. S., SETHI, R., ULLMAN, J. D. **Compiladores: princípios, técnicas e ferramentas**. 2ª ed. São Paulo: Pearson Addison-Wesley, 2008.
- LOUDEN, K. C. **Compiladores: princípios e práticas**. São Paulo: Centage Learning, 2004.